PAP 2 – Python-Auswertung

Wichtige Funktionen für das PAP.

Dieses Dokument ist von mir (FEZ) selbstständig geschreiben und verfasst worden. Viele Funktionen sind von anderen übernommen und abgewandelt worden. Da der Code in VS Code geschrieben wird, wird teilweise auf den MS Copilot (K.I. Modell) zurückgegriffen. Dieser wird unterstützend und nicht generierend benutzt. Ich entschuldige mit hiermit auch formal für das unglaublich schlimme Denglisch in den Kommentaren und der Dokumentation.

Vorlage zum bestimmen aller im Praktikum benötigten Formel und intigriertem Latex Export für eine effizientere Nutzung.

Diese Version ist noch nicht aktiell. Daher führen viele Überschriften in Leere Kapitel. (Stand 07.03.2026)

Inhaltsverzeichnis für den Papulator

- Definition der Versuchsvariablen
- Import aller genutzen Libaries
- Funktionen der Fehlerrechnung
  - Runden signifikanter Stellen
  - Gaussische Fehlerfortpflanzung
  - Berechnung von Messwert und Fehler
  - Signifikante Standartabweichung zweier Messwerte
- Messdatenimporte
  - Patterns
  - Import von Experimentdaten
- Latex Export
  - Export von Gleichungen
  - Export von Einzelergebnissen
  - Export von Messreihen (Tabelle))
- Plotting
- Ergebnisse der Aufgaben

Definition der Versuchsvariablen

Im Folgenden sind Versuchsvariablen definiert, die für einen besseren Workflow sorgen sollen. Diese werden zum exportieren und überschreiben von Dateien wichtig sein und ermöglichen es, diese Datei für jeden Versuch zu benutzen und lediglich die Variablen zu verändern. Es ist jedoch empfohlen eine Kopie der Forlage für jeden Versuch zu machen, damit dieses Dokument strukturiert bleibt.

Zudem sind wichtige Konstanten definiert, die immer wieder auftauchen.

In [5]:
```
versuchsnummer = "223"
versuchsname = "Beispielhafter-Name" # Auf Leerzeichen verzichten
aufgabe = "0z"

# Wichtige Konstanten
# Heidelberger Ortsgravitationsbeschleunigung in m/s^2
g_hd = 9.80984
err_g_hd = 0.00002

# Farben in hex
gruen="#1b722f"
hellgruen = "#1ba97f"
rot = "#b92d3a"
```

```
weinrot="#721c24"
blau="#073984"
lila = "#6610f2"
```

Import aller genutzen Libaries

In [6]:
```
# Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

from decimal import Decimal, ROUND_HALF_UP, getcontext # Besonders für sig. Runden
import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
from scipy.optimize import curve_fit
from scipy.stats import norm
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelmin, argrelmax
from scipy.special import factorial
from scipy.integrate import quad

import matplotlib.pyplot as plt
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path

import pandas as pd # Auch wichtig für den Latex Export
import csv
import re
from typing import List


# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars

# Display und Output
from IPython.display import display, Math, Latex, HTML
```

Funktionen der Fehlerrechnung

Folglich werden eigene Funktionen definiert, die das Auswerten von Versuchen enorm vereinfachen sollen. Diese sind in vielen Punkten an fremden Code orientiert.

Runden signifikanter Stellen

In [7]:
```
def round_sig_digs(val, errVal):
    """
    Funktion zur Rundung eines Fehlers und die Anpassung des Messwertes daran. Diese Funktion wurde etwas umständlicher
    geschrieben, da python mit Floats und Runden schnell in Probleme rennt. Daher musste hier mit dezimal gearbeitet werden.
    Zudem sollten besonders kleine und große Messwerte in der Dezimalschreibweise geschrieben werden, damit diese auch
    für Protokolle geiegnet sind.

    Parameter
    ----------
    **val** : float
        Messwert

    **errVal** : float
        Ungenauigkeit des Messwertes

    Return
```

```
    ------
    **value_rounded** : str
        Gerundeter Messwert

    **error_rounded** : str
        Gerundete Ungenauigkeit des Messwertes

    **res** : str
        "Messwert \\pm Fehler"
    """

    # Daten zu Dezimal wechseln, da Floats probleme machen
    val = Decimal(str(val))
    errVal = Decimal(str(errVal))

    exp = int(math.floor(math.log10(float(errVal))))

    if round(float(errVal / (Decimal(10) ** exp))) < 3:
        exp -= 1

    scale = Decimal(10) ** exp

    val_round = (val / scale).quantize(Decimal('1'), rounding=ROUND_HALF_UP) * scale
    err_round = (errVal / scale).quantize(Decimal('1'), rounding=ROUND_HALF_UP) * scale

    # Exponent des Messwerts bestimmen
    if val_round != 0:
        val_exp = int(math.floor(math.log10(abs(float(val_round)))))
    else:
        val_exp = 0

    # wissenschaftliche Schreibweise, Werte auf selben Exponenten
    if val_exp <= -1:
        scale = Decimal(10) ** val_exp
        v = (val_round / scale)
        e = (err_round / scale)
        digits = -exp + val_exp
        res = f"({v:.{digits}f} \\pm {e:.{digits}f}) e{val_exp}"
    else:
        if exp > 0:
            res = f"{val_round} \\pm {err_round}"
        else:
            digits = -exp
            res = f"{(val_round):.{digits}g} \\pm {(err_round):.{digits}g}"
    return val_round, err_round, res
```

Gaussische Fehlerfortpflanzung

In [8]:
```
def gff(func, errPronePar):
    """
    Kann die Fehlerformel einer gegebenen Gleichung bestimmen.

    Parameters
    ----------
    **func** : sympy function
        Funktion dessen Fehler bestimmt werden soll.

    **errPronePar** : Array
        Liste (Array) aller fehlerbehafteten Größen der Gleichung con sp.Symbols
        Diese Werte werden als x_sym, y_sym, z_sym etc. bezeichnet und sind ungleich den Werten für x, y, z.
        Für die Werte wird daher die Bezeichnung x_val, y_val, z_val etc. genutzt und für deren Fehler err_x, err_y, err_z etc.

    Return
    ----------
    **absolut_err** : sympy function
        Gibt die Fehlergleichung des absoluten Fehlers wieder.

    **relativ_err** : sympy function
        Gibt die Fehlergleichung des relativen Fehlers wieder.

    **errProneParamters** : array
        Liste aller Fehlerbehafteten Größen
    """

    error = 0
    errProneParamters = []
    for errPar in errPronePar:
        d = sp.symbols('d' + errPar.name)
        partial = sp.diff(func, errPar) * d  # Die Funktion wird nach der fehlerbehafteten Variable abgeleitet
        error = error + partial**2 # Fehler werden quadratisch aufsummiert
        errProneParamters.append((errPar,d))
```

```
    absolut_err=sp.simplify(sp.sqrt(error),rational = True)
    relativ_err=sp.simplify(sp.sqrt(error/func**2),rational = True)

    # Debugging Print
    print(f"Die Fehlerfunktion zur Ausgangsfunktion {func} wurde erfolgreich nach den Fehlergrößen {errPronePar} bestimmt.")

    return absolut_err, relativ_err, errProneParamters
```

Berechnung von Messwert und Fehler

In [9]:
```
def calc_with_err(func, errFunc, values):
    """
    Methode zum berechnen von Werten und deren Fehler.

    Parameter
    ----------
    **func** : sympy function
         Funktionen mit Parametern.

    **errFunc** : sympy function
        Die zu func gehörende Fehlerformel. Wird über gff bestimmt.

    **values** :
        Werte, die in die Funktionen eingesetzt werden.
        Als array von Tupeln der Form [(a,da),(b,db),...] oder als array/liste [a,da,b,db,...]
        (Reihenfolge muss die sein, in der die Argumente in der Funktion genommen werden)

    Return
    ----------

    """

    #Falls der Input in mehrere Tupel aufgeteilt ist, werden diese zu einem Array zusammengefügt
    if (np.ndim(values) != 1):
        values = np.concatenate(values)
    result = func(*values[::2])
    uncertainty = errFunc(*values)

    # Debigging Print
    print(f"Die Ergebnisse wurden erfolgreich durch Funktion und Fehlerfunktion bestimmt!")

    return result, uncertainty
```

Signifikante Standartabweichung zweier Messwerte

In [10]:
```
def sigma_abweichung(p1, p2, err_p1 = 0, err_p2 = 0):
    """
    Funktion zum berechnen der Sigma-Abweichugn von zwei Messwerten, oder einem Messwert und einem Literaturwert.
    """
    if err_p1 == err_p2 == 0:
        print("Für die Sigma-Abweichung muss mindestens ein Wert fehlerbehaftet sein!")
    else:
        abweichung = abs(p1 - p2)/(np.sqrt(err_p1 ** 2, err_p2 ** 2))
    return abweichung
```

Messdatenimporte

Für die Auswertung von Versuchen ist es nicht selten, dass Messwerte in *CSV* oder in *TXT* Dateien gespeichert sind. Daher sollen diese Vernünftig eingelesen und verarbeitet werden, um das Benutzen der Daten möglichst einfach und instuinktiv zu gestalten.

Um das Handeln von CSV Tabellen möglichst einfach zu gestalten, werden RIGEX-Pattern genutzt. Das heißt, wenn die Tabelle vor einem Versuch ordentlich strukturiert ist, können die Werte sofort genutzt werden, so wie es gewünscht ist. Dabei sollen folgende Eigenschaften erfüllt sein:

- Das Nutzen einer Kopfzeile ist nicht nur möglich, sondern explizit empfohlen.
- Erkennung eines Messwertes, seiner Einheit und einer Potenz. Das hat dan bspw. die Form: *(name [unit] 10^{power})*
- Der zum Messwert gehörende Fehler. Dies ist meist ein Ablese Fehler. Die Bennenung sollte dabei: *err name* sein.
- Direkte Auswerten von Messreihen. Soll bspw. über mehrere Werte gemittelt werden, so wird automatisiert Mittelwert, Standartabweichung und gesamt Fehler der Messreihe bestimmt.
- Dimensionslose Größen sollen erkannt werden.
- Einstellung eines Indexes. Dies kann entweder die nummerische Angabe des Messdurchganges sein, oder der Parameter-Name. Wird bspw. dieselbe

Messung in Wasser und in Luft durchgeführt, so sollen dies die Parameter-Namen sein. Diese sollen in der ersten Spalte der Tabelle stehen.

Patterns

In [11]:
```
# RegEx Patterns, um autoamtisiert Schemata anzuwenden und Daten anhand dieser auszuwerten.

# Finden eines Messwertes, seiner Einheit und seiner Potenz
pattern = re.compile(
        r'^\s*'
        r'(?P<name>[^\[]+?)'    # alles bis zur ersten '['      # Name
        r'\s*\[\s*'             # echtes '['
        r'(?P<unit>[^\]]+?)'    # alles bis zur ']'             # Einheit
        r'\s*\]'                # echtes ']'
        r'(?:\s*10\^(?P<power>-?\d+))?'                         # optionale 10^n
        r'\s*$'
    )

series_pattern = series_pattern = re.compile(
        r'^(?P<base>\w+)_\d+'
        r'(?:\s*\[\s*(?P<unit>[^\]]+)\s*\])?'
    )

# Index Spalte idx
index_pattern = re.compile(
    r'idx\s*'
    )

# Spalte die Ignoriert werden sill ign
ignore_pattern = re.compile(
    r'(?:ign[^,]*\s*,\s*)?'
)

# Messwert mit Einheit
messurment_pattern = re.compile(
    r'(?P<value>Messwert)\s*$$(?P<unit>[^$$]+)\]\s*,\s*'
)

# Fehler des Messwertes
error_pattern = re.compile(
    r'(?:err\s+Messwert\s*,\s*)?'
)
```

Import von Experimentdaten

Das einlesen von Messweerten und das speichern dieser als Arrays für einen besseren zugriff. Automatisches herausfinden von Messwert und dazugehörigem Fehler.

In [12]:
```
def import_experimental_data(path:str = versuchsnummer + ".csv", setDelimiter:str = ",", setHeader:int = 0, setIndex_col:int = None):
    """

    Parameter
    ---------
    **path**: str
        Relatives verzeichnes der einzulesenden Datei. Am besten CSV-Datei einfach unter *versuchsnummer.csv* im Python-Folder speicher

    **setDelimiter**: str
        Default ist hier das Komma:  > , <

    **setHeader**: int
        Setzt fest, was die Header-Row ist. Default ist 0 (die oberste Row)

    Return
    ---------
    **cleanded** : Pandas DataFrame
        DataFrame, welches sicher zum weiterverwenden ist. NaN und ähnliches werden herausgefiltert und per Print
    """

    # Liest die Daten unmaipuliert
    try:
        df = pd.read_csv(path, delimiter=setDelimiter, header=0, index_col=None)
    except FileNotFoundError:
        print(f"The file {path} was not found.")

    # Wir erwarten nicht, dass zwei perfekt identische Zeilen existieren koennten, daher werden alle identischen Zeilen geloescht. Somi
    # Langfristig soll das noch verschönert werden und wirklich nur NaN Zeilen gelöscht werden
    no_NaN_data = df.drop_duplicates(keep= False)
    df.reset_index(drop=True, inplace=True)
```

```
    return no_NaN_data

# Speichert direkt das DataFrame als Variable, so kann dieses direkt in anderen Cells benutzt werden.
# experimental_data = import_experimental_data()
```

Latex Export

Erstellen der dedizierten Dartei zum Speichern der Werte für **LATEXT**

Folgender Code sollte einmal ausgeführt werden, damit direkt die Datei zum Speichern erstellt werden kann. Unbedingt daran denken, die Versuchsnummer anzupassen, die Werte anderer Versuche könnten ansonsten verloren gehen. Es gibt jedoch eine sicherheits Kopie.

Die File wird Folgendes beinhalten:

- Formel, sowie Formel nach gff
- Berechnete Werte und deren Fehler
  - Wert + Fehler
  - Tabellen Export

In [13]:
```
def create_tex_result_file(fileName:str = f"python-results.sty", relativePath:str="../Auswertung/"):
    """
    Diese Method erstellt automatisch eine tex-Datei, in dem Messwerte bzw. deren Ergebnisse, Tabellen weiteres als variable gespeicher
    Die File wird unter *Versuche/${versuchsnummer}$/Auswertung/python-results.tex* zufinden sein. Am einfachsten ist es jedoch die Par
    angepasst werden muss.

    Die Python-File muss im Python-Ordner des jeweiligen Versuches liegen!

    Parameter
    ----------
    **fileName** :
        neuer Name, falls die Datei besonders heißen soll.

    Return
    --------
    **path** : str
        Gibt den Path zur file wieder, welche autoamtisch generiert. In diese sollen später die Formeln geschrieben werden.
    """

    path = relativePath + fileName

    if os.path.isfile(path):
        print(f"{fileName} existiert bereits.")
        pass
    else:
        print(f"{fileName} wird in {relativePath} erzeugt.")
        with open(path, 'w') as file:
            file.write("% Dies ist eine automatisch generierte Datei. Hier werden automatisiert Variablen fuer Formeln, Ergebnisse und

        with open(path + "-BackUp.sty", 'w') as bFile:
            bFile.write("% Dies ist eine automatisch generierte Datei. Hier wird dediziert ein Back-Up erstellt, damit Werte nicht verl

    return path

# pyPath ist die File, in die der Latex Export geschehen soll.
pyPath = create_tex_result_file()
```

```
python-results.sty existiert bereits.
```

Export von Gleichungen

In [14]:
```
def function_to_latex(func, texVarName:str, texCom:str):
    """
    Zeigt die Formel als gerenderte Math-Darstellung und darunter
    den Latex-Quelltext, der per Button kopiert werden kann. (Für leichtere Benutzung als HTML).

    Zudem wird die Latexformel als Variable in Latex gespeichert.

    Parameters
    ----------
    **f** : sympy function

    **texVarName** : str
        Einzigartiger Name der Variablen. Dieser wird zum überschreiben alter Formel gebraucht.
```

```
    **texCom** : str
        setzt den newcommand-Kürzel für latex fest. Setzte kein Backslash! Auch texCom muss einzigartig gesetzt werden.
    """
    print("Die gegebene Funktion lautet: \n")
    display(Math(sp.latex(func, long_frac_ratio=2).replace('thisWillTurnDelta', r'\Delta ')))

    print("Hier ist der dazugehörige Latex code: \n")
    latex_str = sp.latex(func, long_frac_ratio=2).replace('thisWillTurnDelta', r'\Delta ')
    html = f"""
    <div style="margin-top:0.5em;">
        <code id="latex-code-{id(func)}">
            {latex_str}
        </code>
        <button onclick="
            const tex_as_txt = document.getElementById('latex-code-{id(func)}').innerText;
            navigator.clipboard.writeText(tex_as_txt)
        " style="
            margin-left:8px;
            padding:2px 6px;
            cursor:pointer;
        ">
            Kopieren
        </button>
    </div>
    """
    display(HTML(html))


    # Hinzufuegen bzw. Ueberschreieben der Formel in die Sammlung
    with open(pyPath, 'r') as file:
        lines = file.readlines()
    found = False

    with open(pyPath, 'r') as file:
        for lineNum,  line in enumerate(lines, 1):
            if texVarName in line: # Checkt, ob der Variablen Name bereits vergeben ist.
                lines[lineNum] = "\\newcommand{\\" + texCom + "}{" + latex_str + "} \n\n"
                found = True
                # Deugging print
                # print(f'{texVarName} is at line {lineNum}')

    if not found:
        print("Die neue Funktion wurde hinzugefügt")
        with open(pyPath, 'w') as file:
            file.writelines(lines) # Schreibt den alten Stand
            file.write("\n\n% " + texVarName + "\n") # Fügt die Variable als Kommentar hinzu
            file.write("\\newcommand{\\" + texCom + "}{" + latex_str + "} \n\n")
    else:
        print("Die alte Funktion wurde erfolgreich überschrieben.")
        with open(pyPath, 'w') as file:
            file.writelines(lines)
```

Export von Einzelergebnissen

Export von Messreihen (Tabelle)

In [15]:
```
def table_to_latex(df_tex: str, texVarName: str, texCom: str):
    """
    Fügt aus einem DataFrame entstandene Latex Tabelle der special Python File hinzu.

    Parameters
    ----------
    **df_tex** : pd.DataFrame.to_latex
        Uebersetztes DataFrame. Kann Label und Caption handlen. Dezimaltrennung steht auf ','.

    **texVarName** : str
        Einzigartiger Name der Variablen. Dieser wird zum überschreiben alter Formel gebraucht.

    **texCom** : str
        setzt den newcommand-Kürzel für latex fest. Setzte kein Backslash! Auch texCom muss einzigartig gesetzt werden.

    Returns
    -------
    Schreibt automatisch in die sty file
    """

    # Hinzufuegen bzw. Ueberschreieben der Tabelle in die Sammlung
    with open(pyPath, 'r') as file:
```

```
        lines: List[str] = file.readlines()
    found = False

    with open(pyPath, 'r') as file:
        for lineNum,  line in enumerate(lines):
            if texVarName in line: # Checkt, ob der Variablen Name bereits vergeben ist.
                print(f'{texVarName} starts at line {lineNum + 1}')
                found = True
                table_start_line = lineNum
                break

    if not found:
        print(f"Die neue Tabelle ({texVarName}) wurde hinzugefügt")
        with open(pyPath, 'w') as file:
            file.writelines(lines) # Schreibt den alten Stand
            file.write("\n% " + texVarName + "\n")
            file.write("\\newcommand{\\" + texCom + "}{" + df_tex + "}\n\n")
    else:
        print(f"Die alte Tabelle ({texVarName}) wird überschrieben.")
        start_table = table_start_line + 1
        while start_table < len(lines) and lines[start_table].strip() == "":
            # Leere Zeile überspringen - das ist unser Marker
            start_table += 1
            break   # Wir wollen nur die erste leere Zeile

        # Falls kein leerer Marker gefunden wurde, gehen wir davon aus,
        # dass die Tabelle direkt in der nächsten Zeile startet.
        if start_table >= len(lines):
            start_table = table_start_line + 1

        end_table = start_table

        while end_table < len(lines):
            cur = lines[end_table].strip()

            # Abbruchbedingungen:
            if (
                cur.startswith(r"\\newcommand")
                or cur.startswith(r"\\section")
                or cur.startswith(r"\\subsection")
                or cur.startswith(r"\\begin")
            ):
                break

            end_table += 1

        if end_table > start_table:
            del lines[start_table - 1:end_table]

        insertion_point = table_start_line + 1  # Direkt nach der Zeile mit texVarName
        new_block = [
            # "\n\n",
            f"% {texVarName}\n",
            f"\\newcommand{{\\{texCom}}}{{{df_tex}}}\n",
        ]
        lines[insertion_point:insertion_point] = new_block

        with open(pyPath, 'w') as file:
            file.writelines(lines)
```

Finaler Export

Für die Auswertung sollen Messwerte und Fehler in einer Zelle stehen. Dies tut *tab_merge_val_err* automatisiert.

In [16]:
```
def tab_merge_val_err(df: pd.DataFrame, show_series_values:bool=False, setTexTabCap: str="", setTexTabLab:str=""):
    """Sucht im DataFrame nach einem Asdruck:
        "<name> [<unit>] <power>"
    und dem dazugehörigem Fehlerausdruck:
        "err <name>".
    Erstellt ein neues DataFrame für den Latex-Export

    Parameters
    ----------
    **df* : pd.DataFrame
        Original data frame.

    **show_series_values** : bool
        Rather the table should show the experimental data or just its mean


    Returns
```

```
    -------
    pd.DataFrame
        Eine Kopie von ``df``.
    """

    # Kopiert das alte DataFrame
    df_tex = pd.DataFrame(index=df.index)


    def format_tex(row):
        val = row[col]
        err = row[err_col]
        # return f'({val} \\pm {err}) \\mathrm{{{header_unit}}}'
        val_r, err_r = round_sig_digs(err, val)
        return f'${val_r} \\pm {err_r}$'

    # ----------------------------
    # 1) Einzelmessungen
    # ----------------------------

    for col in df.columns:
        col_match = pattern.match(col)

        if not col_match:
            # No header that follows the expected schema – skip it
            # print(f'Skipping column {col!r}: not "<value> [<unit>] <power>"')
            continue

        # Extract the captured parts
        name = col_match.group('name').strip()
        unit = col_match.group('unit').strip()
        power = col_match.group("power")
        # print(f'Found measurement column: name={name!r}, unit={unit!r}, power={power!r}')

        # Build the name of the error column that must exist
        err_col = f'err {name}'
        if err_col not in df.columns:
            # print(f'No error column "{err_col}" for measurement "{name}" - skipping')
            continue

        if power:
            header_unit = f"$\\mathrm{{{unit}}} \\cdot 10^{{{power}}}$"
        else:
            header_unit = f"$\\mathrm{{{unit}}}$"

        df_tex[f'{name} [{header_unit}]'] = df.apply(format_tex, axis=1)


    # ----------------------------
    # 2) Messreihen
    # ----------------------------
    grouped = {}

    for col in df.columns:
        series_match = series_pattern.match(col)
        if series_match:
            base = series_match.group("base")
            unit = series_match.group("unit")
            # power = col_match.group("power")
            grouped.setdefault(base, {"cols": [], "unit": unit})
            grouped[base]["cols"].append(col)


    # --- Berechnungen ---
    for base, info in grouped.items():
        cols = info["cols"]
        series_unit = info["unit"] or ""


        err_col = f"err {base}"
        if err_col not in df.columns:
            continue

        values = df[cols]

        if show_series_values == True:
            # Einzelwerte in DataFrame hinzufuegen
            for ind, col in enumerate(cols, start=1):
                df_tex[f'{base}_{ind} [{unit}]'] = df.apply(format_tex, axis=1)

        # Mittelwert
        mean = values.mean(axis=1)
```

```
        # Standardabweichung
        std = values.std(axis=1, ddof=1)

        # Fehler des Mittelwerts
        stat_err = std / np.sqrt(len(cols))

        # systematischer Fehler
        sys_err = df[err_col]

        # Gesamtfehler
        total_err = np.sqrt(stat_err**2 + sys_err**2)

        # if power:
        #     header_unit = f"\\mathrm{{{unit}}} \\cdot 10^{{{power}}}"
        # else:
        #     header_unit = f"\\mathrm{{{unit}}}"

        formatted = []

        for m, e in zip(mean, total_err):
            m_r, e_r = round_sig_digs(e, m)
            formatted.append(f"${m_r} \\pm {e_r}$")

        df_tex[f"$\\overline{{{base}}} [\\mathrm{{{series_unit}}}]$"] = formatted

    setTexTabCap="Dies ist die Caption der Tabelle"
    setTexTabLab="bspTab"

    table_to_latex(df_tex.to_latex(decimal=',', caption=setTexTabCap, label=setTexTabLab, position="h", index=False), texVarName="Beisp

    print(df_tex.to_latex(decimal=',', caption=setTexTabCap, label=setTexTabLab, position="h"))
    return df_tex.to_latex(decimal=',', caption=setTexTabCap, label=setTexTabLab, position="h")


# combine_value_error(clean_data)
```

Berechnen neuer Spalten mit Fehler

Die folgende Funktion berechnet anhand einer gegebenen Formel und den gegebenen Parametern die Ergebnisse einer neuen Spalte mit Ihren Fehlern.

In [17]:
```
def add_column_by_formula(func, errProneParams, headerName:str, headerUnit:str, err_free_params = []):
    pass
```

Plotting

Ergebnisse der Aufgaben

Lesen der Text Datei

In [ ]:
```
t,x,y = np.loadtxt("position_data.txt", delimiter=",", dtype="float",
                   unpack=True)

# t,x,y = np.loadtxt("geklaut.txt", delimiter=",", dtype="float",
#                    unpack=True)

# t,x,y = np.loadtxt("FuN.txt", delimiter=",", dtype="float",
#                    unpack=True)
```

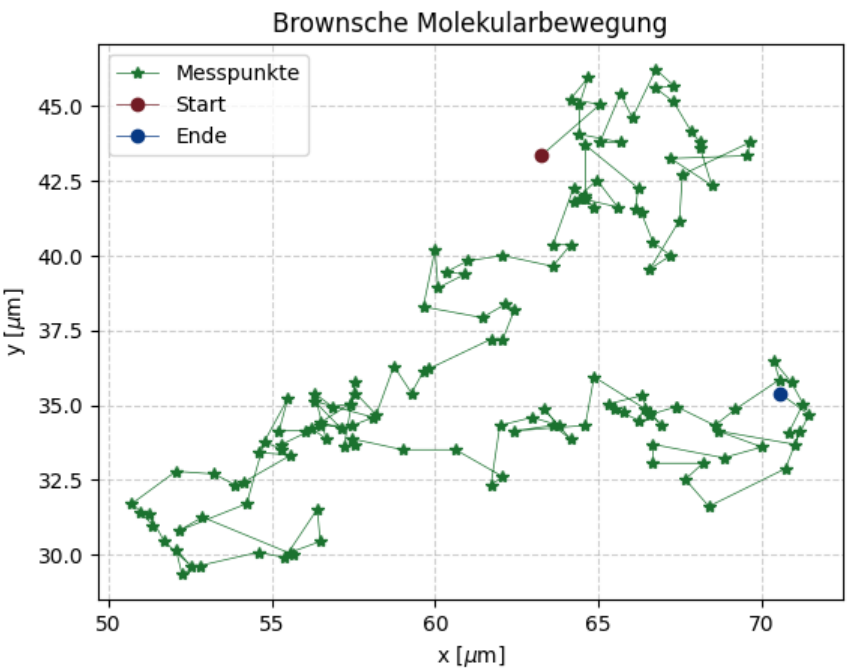
In [19]:
```
# Speiherort
pfad = "../img/FuN/"
```

Aufgabe 1

In [20]:
```
aufgabe = "1a"
```

In [21]:
```
# Messpunkte
plt.plot(
```

```
    x,
    y,
    marker='*',
    color=gruen,
    linewidth=0.5,
    label="Messpunkte"
    )

# Start
plt.plot(
    x[0],
    y[0],
    marker='o',
    color=weinrot,
    linewidth=0.5,
    label="Start"
    )

plt.plot(
    x[149],
    y[149],
    marker='o',
    color=blau,
    linewidth=0.5,
    label="Ende"
    )


plt.xlabel(r'x [$\mu$m]')
plt.ylabel(r'y [$\mu$m]')
plt.title('Brownsche Molekularbewegung')
plt.grid(True, which="both", linestyle="--", alpha=0.6)

plt.legend()

plt.savefig(f'{pfad}Bewegung-{aufgabe}.pdf', dpi=300)
plt.show()
```



Aufgabe 2

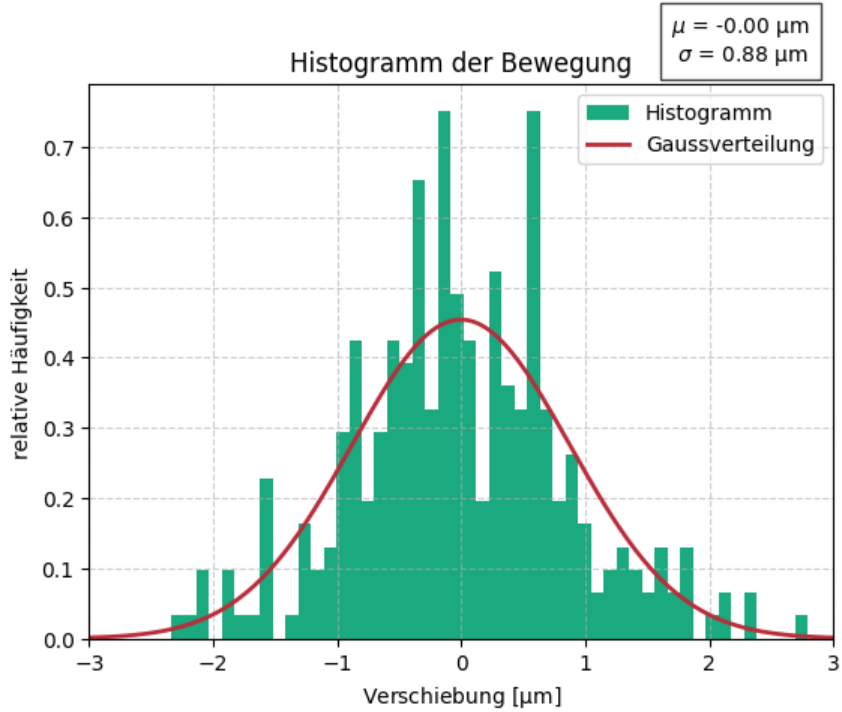
In [22]:
```
dt=np.array([])
dx=np.array([])
dy=np.array([])

i=0
while i < len(t)-1:
    dt=np.append(dt,t[i+1]-t[i])
    dx=np.append(dx,x[i+1]-x[i])
    dy=np.append(dy,y[i+1]-y[i])
    i = i + 1

r_squared= dx**2 + dy**2
```

```
r_squared_mean=np.mean(r_squared)
print(f"s = {r_squared_mean:.3f} micro m^2")

r_squared_mean_std=np.std(r_squared)/np.sqrt(len(r_squared))
print(f"r_squared_mean_std = {r_squared_mean_std:.3f}")

dt_mean=np.mean(dt)
print(f"dt_mean = {dt_mean:.3f}s")
```

```
s = 1.544 micro m^2
r_squared_mean_std = 0.137
dt_mean = 1.000s
```

In [23]:

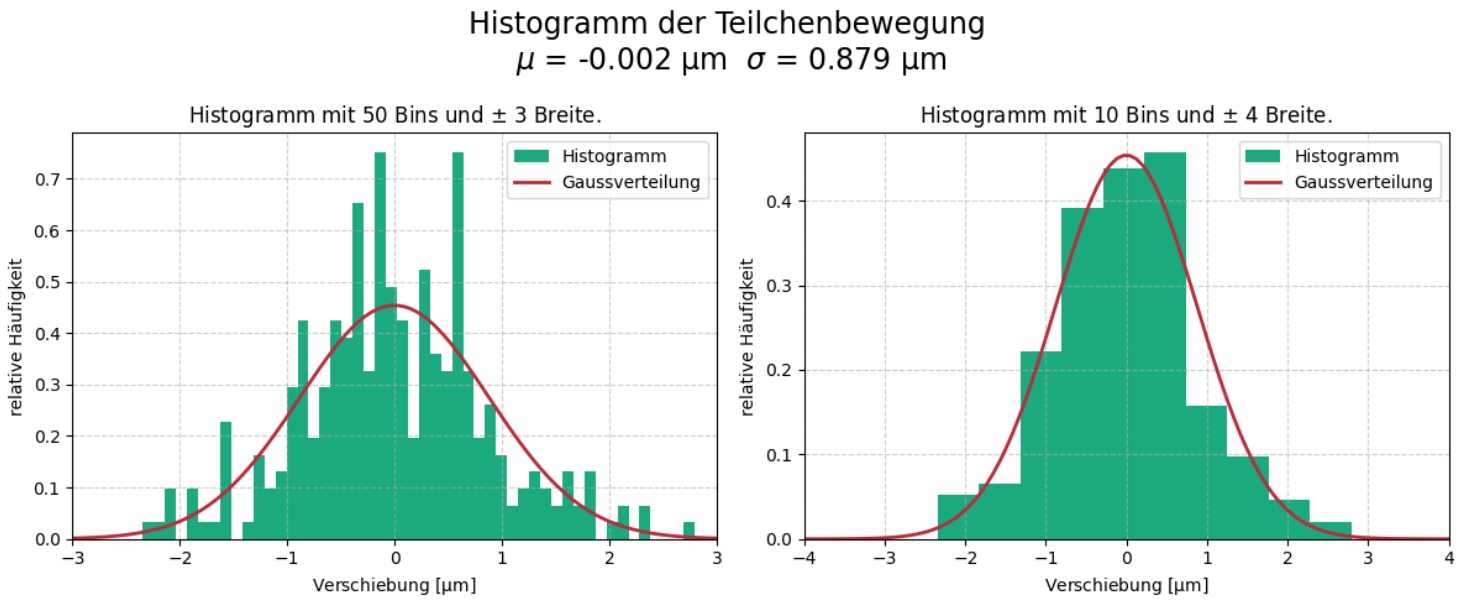
```
aufgabe = "2a"

all_data=np.append(dx,dy)
plt.hist(
    all_data,
    density=True,
    bins = 50,
    color=hellgruen,
    label="Histogramm"
    )

mu_val=np.mean(all_data)
sigma=np.std(all_data)

text_tol = f'$\\mu$ = {mu_val:.2f} ' + '$\\mathrm{\\mu m}$' + f'\n $\\sigma$ = {sigma:.2f} ' + '$\\mathrm{\\mu m}$'

# print(f'Mittelwert: {mu_val:.3f}')
# print(f'Standardabweichung {sigma:.3f}')

gauss = norm.pdf(
            np.linspace(-3,3,num=200),
            mu_val,
            sigma
            )

plt.plot(
    np.linspace(-3,3,num=200),
    gauss,
    '-',
    linewidth=2,
    color=rot,
    label="Gaussverteilung"
    )

plt.xlim(-3,3)
plt.title("Histogramm der Bewegung")
plt.xlabel(r'Verschiebung [$\mathrm{\mu}$m]')
plt.ylabel('relative Häufigkeit')
plt.grid(True, which="both", linestyle="--", alpha=0.6)

plt.legend()
plt.text(2.25, 0.85, text_tol, horizontalalignment='center',
     verticalalignment='center', bbox={'facecolor': 'w', 'alpha': 0.75, 'pad': 5})

plt.savefig(f'{pfad}Histogramm-{aufgabe}.pdf', dpi=300)
plt.show()
```



In [36]:

```
aufgabe = "2b"
# Gauss
fig, ax = plt.subplots(1, 2, figsize=(12,5))

all_data = np.append(dx, dy)

mu_val = np.mean(all_data)
sigma = np.std(all_data)

text_tol = f'$\\mu$ = {mu_val:.3f} ' + '$\\mathrm{\\mu m}$  ' + f'$  \\sigma$ = {sigma:.3f} ' + '$\\mathrm{\\mu m}$'

x = np.linspace(-3,3,100)
gauss = norm.pdf(x, mu_val, sigma)

# Linker Plot
ax[0].hist(
    all_data,
    density=True,
    bins=50,
    color=hellgruen,
    label="Histogramm"
)

ax[0].plot(
    x,
    gauss,
    '-',
    linewidth=2,
    color=rot,
    label="Gaussverteilung"
)

ax[0].set_xlim(-3,3)
ax[0].set_title("Histogramm mit 50 Bins und $\\pm$ 3 Breite.")
ax[0].set_xlabel(r'Verschiebung [$\mathrm{\mu}$m]')
ax[0].set_ylabel('relative Häufigkeit')
ax[0].grid(True, which="both", linestyle="--", alpha=0.6)
ax[0].legend()


# Rechter Plot

x = np.linspace(-4,4,100)
gauss = norm.pdf(x, mu_val, sigma)

ax[1].hist(
    all_data,
    density=True,
    bins=10,
    color=hellgruen,
    label="Histogramm"
)
```

```
ax[1].plot(
    x,
    gauss,
    '-',
    linewidth=2,
    color=rot,
    label="Gaussverteilung"
)

ax[1].set_xlim(-4,4)
ax[1].set_title("Histogramm mit 10 Bins und $\\pm$ 4 Breite.")
ax[1].set_xlabel(r'Verschiebung [$\mathrm{\mu}$m]')
ax[1].set_ylabel('relative Häufigkeit')
ax[1].grid(True, which="both", linestyle="--", alpha=0.6)
ax[1].legend()


plt.suptitle(f"Histogramm der Teilchenbewegung \n" + text_tol, size=17)

plt.tight_layout()

plt.savefig(f'{pfad}Histogramm-{aufgabe}.pdf', dpi=300)
plt.show()
```

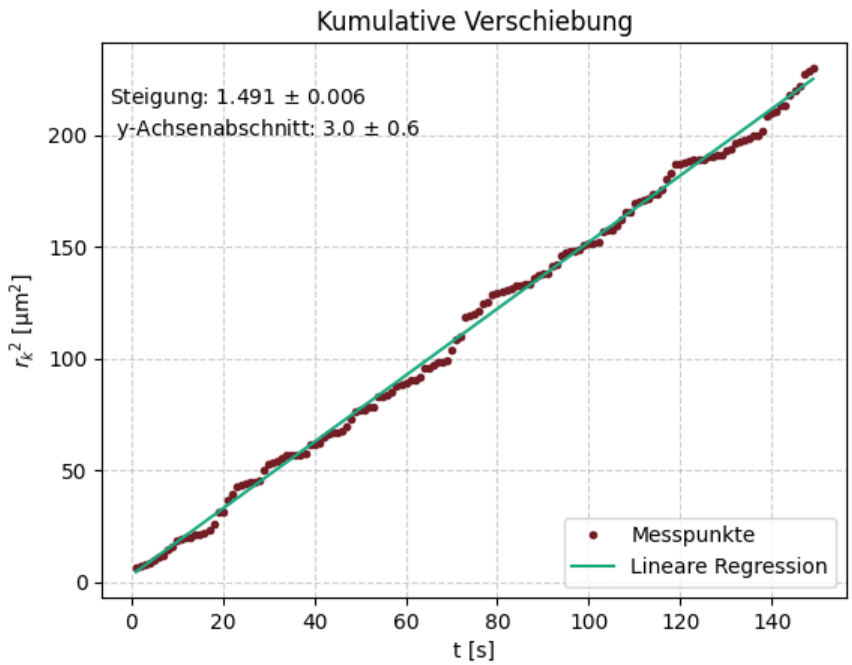


Aufgabe 3

In [34]:
```
aufgabe = "3a"

r_kumm=np.cumsum(r_squared)

def linear(x,a,b):
    return a*x+b

popt, pcov = curve_fit(linear, t[1:], r_kumm)

steig = round_sig_digs(popt[0], np.sqrt(pcov[0,0]))
absch = round_sig_digs(popt[1], np.sqrt(pcov[1,1]))

print(f"Steigung: {steig[2]} \n", f"y-Achsenabschnitt: {absch[2]}")


plt.plot(
    t[1:],
    r_kumm,
    marker='.',
    color=weinrot,
    linewidth=0,
    label="Messpunkte")

plt.plot(
    t[1:],
    linear(t[1:],*popt),
    color=hellgruen,
    label="Lineare Regression")
```

```
plt.xlabel('t [s]')
plt.ylabel(r'${r_k}^2$ [$\mathrm{\mu m^2}$]')
plt.title('Kumulative Verschiebung')
plt.grid(True, which="both", linestyle="--", alpha=0.6)
plt.legend()
plt.text(-5, 200, f"Steigung: {steig[0]} $\\pm$ {steig[1]} \n y-Achsenabschnitt: {absch[0]} $\\pm$ {absch[1]}")

plt.savefig(f'{pfad}kumulativ_{aufgabe}.pdf')
plt.show()
```

```
Steigung: 1.49 \pm 0.006
 y-Achsenabschnitt: 3 \pm 0.6
```

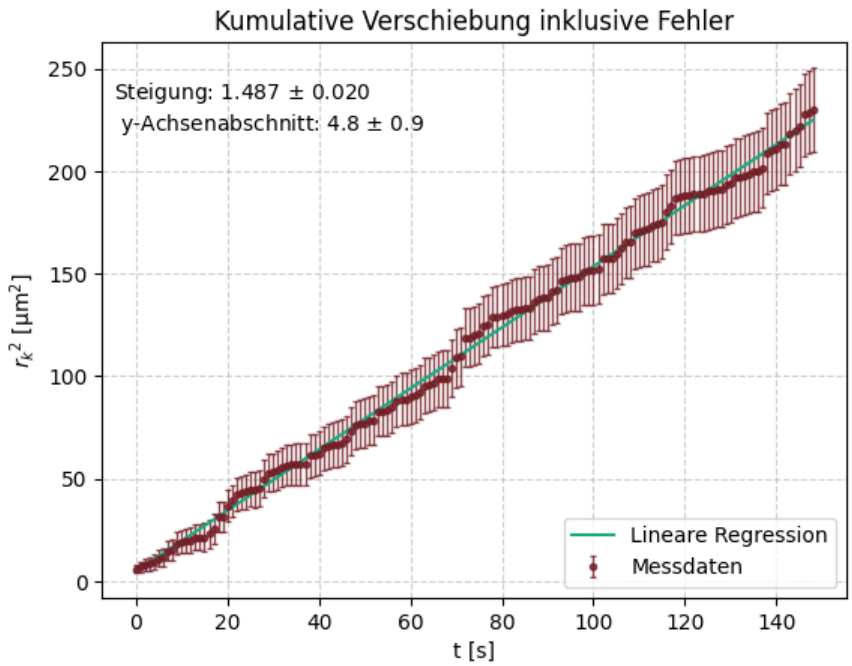


In [37]:

```
aufgabe = "3b"
# Kumulierte Verschiebung
r_kumm=np.cumsum(r_squared)

# Fehler der kumulierten Verschiebung
sigma_r2 = np.std(r_squared)
n = np.arange(1, len(r_kumm) + 1)
r_kumm_err = sigma_r2 * np.sqrt(n)

# Lieare Regression
def linear(x,a,b):
    return a*x+b

popt, pcov = curve_fit(linear, t[:-1], r_kumm,sigma = r_kumm_err, absolute_sigma = True)

steig = round_sig_digs(popt[0], np.sqrt(pcov[0,0]))
absch = round_sig_digs(popt[1], np.sqrt(pcov[1,1]))

print(f"Steigung: {steig[2]} \n", f"y-Achsenabschnitt: {absch[2]}")


plt.errorbar(
    t[:-1],
    r_kumm,
    yerr = r_kumm_err,
    fmt='.',
    color=weinrot,
    linewidth=1,
    capsize=1.5,
    alpha = 0.8,
    label = "Messdaten")

plt.plot(
    t[:-1],
    linear(t[:-1],*popt),
    color=hellgruen,
    label="Lineare Regression")


plt.xlabel('t [s]')
plt.ylabel(r'${r_k}^2$ [$\mathrm{\mu m^2}$]')
plt.title('Kumulative Verschiebung inklusive Fehler')
plt.grid(True, which="both", linestyle="--", alpha=0.6)
```

```
plt.legend()
plt.text(-5, 220, f"Steigung: {steig[0]} $\\pm$ {steig[1]} \n y-Achsenabschnitt: {absch[0]} $\\pm$ {absch[1]}")

plt.savefig(f'{pfad}kumulativ_{aufgabe}.pdf')
plt.show()
```

```
Steigung: 1.49 \pm 0.020
 y-Achsenabschnitt: 5 \pm 0.9
```



Aufgabe 4

In [30]:
```
aufgabe = "4a"
```

In [38]:
```
# Radius
a = 378e-9
da = 15e-9

# Temperatur
T = 22.5 + 273.15
dT = 0.5

# Viskosität des Wassers bei gegebener Temperatur
eta  = 9.50 *10**(-4)
err_eta  = 0.1  * 10**(-4)

# Steigung umgerechnet in Meter
m  = popt[0] * 10**(-12)
dm = np.sqrt(pcov[0][0])* 10**(-12)

# Berechnung der Boltzmannkonstante aus den Rohdaten(r^2, t)
kB_r =  6 * np.pi*eta * a * r_squared_mean*1e-12/(4* T*dt_mean)
err_kB_r = np.sqrt((kB_r/a * da)**2 + (kB_r/(r_squared_mean*1e-12) * r_squared_mean_std*1e-12)**2)

# Berechnung der Boltzmannkonstante aus der Approximationsgeradensteigung
kB_m =  6 * np.pi*eta * a/(4* T) * m
err_kB_m =  6 * np.pi*eta * m/(4* T) * da

# Berechnung der Diffusionskoeffizient aus der Approximationsgeradensteigung
D_m = m/4
err_D_m = dm/4

# Berechnung
D_r = r_squared_mean/(4*dt_mean)
err_D_r = r_squared_mean_std/(4*dt_mean)

# Wertevergleiche
kB_l =  1.38064852e-23 # Literaturwert
sig_kb1 = np.abs(kB_l - kB_r)/err_kB_r
sig_kb2 = np.abs(kB_l - kB_m)/err_kB_m
sig_kb3 = np.abs(kB_r - kB_m)/np.sqrt(err_kB_r**2 + err_kB_m**2)

# Runden der Werte für die Boltsmannkonstante auf signifikante Stellen
bolts_steig = round_sig_digs(kB_m, err_kB_m)
bolts_rho = round_sig_digs(kB_r, err_kB_r)
```

```python
print("---------")
print(f"Boltzmannkonstante kB (Rohdaten): kB = ({bolts_rho[2]}) J/K")
print(f"Boltzmannkonstante kB (Steigung): kB = ({bolts_steig[2]}) J/K")

print(f"Werteabweichung (Steigung - Rohdaten): {sig_kb3:.3g}\\sigma")
print(f"Werteabweichung (Literaturwert - Rohdaten): {sig_kb1:.3g}\\sigma")
print(f"Abweichung (Literaturwert - Steigung): {sig_kb2:.3g}\\sigma")

# Berechnung der Diffusionskoeffizient
sig_D = np.abs(D_r - D_m)/np.sqrt(err_D_r**2 + err_D_m**2)

# Runden der Werte für den Diffusionskoeffizienten auf signifikante Stellen

diffus_steig = round_sig_digs(D_m, err_D_m)
diffus_rho = round_sig_digs(D_r, err_D_r)

print("---------")
print(f"Diffusionskeffizient D (Rohdaten): D = {diffus_rho[2]} m^2/s")
print(f"Diffusionskoeffizient D (Steigung): D = {diffus_steig[2]} m^2/s")
print(f"Werteabweichung: {sig_D:.3g}\\sigma")
```

```
---------
Boltzmannkonstante kB (Rohdaten): kB = ((8.8 \pm 0.9) e-24) J/K
Boltzmannkonstante kB (Steigung): kB = ((8.5 \pm 0.3) e-24) J/K
Werteabweichung (Steigung - Rohdaten): 0.358\sigma
Werteabweichung (Literaturwert - Rohdaten): 5.8\sigma
Abweichung (Literaturwert - Steigung): 15.7\sigma
---------
Diffusionskeffizient D (Rohdaten): D = (3.9 \pm 0.3) e-1 m^2/s
Diffusionskoeffizient D (Steigung): D = (3.72 \pm 0.05) e-13 m^2/s
Werteabweichung: 11.3\sigma
```